

1. Problem description

2. Implementation Details

2.2 Download sample CVs and output structurally

```
extraction chain = prompt template | structured llm
```

The diagram illustrates the process of converting raw data from a CV into a structured JSON format. On the left, under the heading "Raw data", is a snippet of a CV for John Smith, a Marketing Professional at ByteDance in Singapore. The CV includes sections for Experience, Education, and Skills. An arrow points from this raw data to the right, where the "Processed data" is shown as a JSON object. This JSON object maps the CV sections to keys: "personal_info" for basic details, "work_experience" for the professional history, "education" for academic background, and "skills" for a list of competencies.

```

{
  "personal_info": {
    "full_name": "John Smith",
    "professional_title": "Marketing Professional",
    "location": "Singapore, Singapore, Kowloon",
    "current_company": "ByteDance",
    "email": null,
    "education": [
      {
        "school": "McGill University",
        "degree": "Bachelor of Science (BSc) in Marketing"
      }
    ]
  },
  "work_experience": [
    {
      "company": "ByteDance",
      "role": "Engineer",
      "start_year": "2020",
      "end_year": "Present"
    }
  ],
  "skills": [
    "Content Creation",
    "SEO",
    "Social Media"
  ]
}

```


Processed data

2.3 Connect to MCP server

We load the MCP tools and equip these tools for LLM.

code 1

```
client = MultiServerMCPClient({
    "social_graph": {
    "transport": "http", "url": "https://ftec5660.ngrok.app/mcp",
    "headers": {"ngrok-skip-browser-warning": "true"} } })
mcp_tools = await client.get_tools()
llm_with_tools = llm.bind_tools(tools)
```

The "tool list" contains 6 tools that can be used to search for the LinkedIn and Facebook IDs and profiles of the candidates.

code 2

```
tools = [
    search facebook users,
    get facebook profile,
    get facebook mutual friends,
    search linkedin people,
    get linkedin profile,
    get linkedin interactions
]
await tools[3].ainvoke({'q':
structured_cvs["CV_2.pdf"]['full_name'], 'limit': 10})
```

2.4 Matching and scoring

In this part, three chains were built in total.

2.4.1 LinkedIn ID search chain

```
li_search_chain = li_match_prompt | li_match_llm
```

The first chain is used to search for the candidate's LinkedIn ID. As the `search linkedin people` tool will return several possible users with the same name. Thus, the LLM was involved in identifying the correct person based on the location, skills and headline.

After obtaining the accurate ID, we can use the ID as a parameter for the `get`

`LinkedIn profile` tool to retrieve a complete professional profile.

2.4.2 Facebook ID Match chain

```
fb_search_chain = fb_match_prompt | fb_match_llm
```

Since the `search facebook users` tool can only search for users based on their display names, and many users' display names may not be their real names. Therefore, using this method, it may not be possible to find the actual account of the corresponding candidate.

However, after conducting numerous experiments, I discovered that a user's Facebook ID and LinkedIn ID are indeed corresponding. To further confirm this, a second chain was set up to verify this phenomenon.

The LinkedIn ID found is used as a parameter for the `get facebook profile` tool to retrieve a complete Facebook profile with personal info and activity. Similarly, the LLM will be involved in verifying whether this person is indeed the corresponding candidate's account by comparing the information on the profile and the CV.

2.4.3 Score chain

```
score_chain = judge_prompt | judge_llm
```

After confirming the candidate's LinkedIn and Facebook IDs, we can officially start to verify the information in their resume.

The maximum score is 1.0, and the rating is divided into three parts: Core Occupation and Educational Experience, Account Authenticity and Social Endorsement, and Auxiliary Verification.

Core Occupation and Educational Experience is the most important dimension for a candidate's resume information. Therefore, I assigned **60%** of the weight to this section. Agent would carefully compare the company names, job titles, employment duration, and schools in the CV and LinkedIn profile.

Note: I find that there are some minor time discrepancies between CV and social media data. However, this is normal for active job seekers as the CVs are static documents. Thus, I incorporated Tolerance time & status discrepancies into the

prompt and ask agent not to treat this as fraud.

Account Authenticity and Social Endorsement account for **20%** of the score. In this part, `get linkedin interactions` tool was used in order to prevent a "zombie account" that was created specifically for creating fake resumes.

Facebook is more of an entertainment-oriented social networking platform, and it may be difficult to directly identify the real corresponding users, so the profile data on Facebook is only used as an auxiliary part (**20%**). So even if the candidate's Facebook account cannot be found, their resume will not be directly rejected

Based on this standard, the agent will detect discrepancies between CV claims and social media data and finally generate a verification report.

3. Task completion

3.1 Final loop

code 3

```
scores=[] score_report={}
for file_name, cv_data in structured_cvs.items():
    candidate_name = cv_data["full_name"]

    li_search_results = await tools[3].ainvoke({'q':
candidate_name, 'limit': 10})

    li_matched_ids = await li_search_chain.ainvoke({ "cv":
cv_data, "li_res": li_search_results, })

    fb_search_results= await tools[1].ainvoke({
"user_id": li_matched_ids.linkedin_id })

    fb_matched_ids = await fb_search_chain.ainvoke({
"cv": cv_data, "fb_res": fb_search_results})

    linkedin_prof = await tools[4].ainvoke({
'person_id': li_matched_ids.linkedin_id})

    linkedin_interact = await tools[5].ainvoke({
'person_id': li_matched_ids.linkedin_id})

    grade_result = await score_chain.ainvoke({
"cv": cv_data, "li_prof": linkedin_prof,
"li_interact": linkedin_interact, "fb_prof":
```

```
fb_search_results, })  
  
scores.append(grade_result.score)  
score_report[file_name] = grade_result.report
```

Finally, through a loop, we instructed the LLM to verify and score each CV separately. The report will provide a detailed analysis of the differences between the resume and the information on social platforms, as well as the reasons for the scores.

3.2 Evaluation code

I ran the evaluation code. The final result was in line with expectations. The agent was able to correctly judge the authenticity of each resume.

code 2

```
scores = [0.74, 1.0, 0.8, 0.2, 0.3]  
groundtruth = [1, 1, 1, 0, 0]  
result = evaluate(scores, groundtruth)  
print(result)
```

```
{'decisions': [1, 1, 1, 0, 0], 'correct': 5, 'total': 5, 'final_score': 1.0}
```